

ASIC 课程项目报告

人体反应速率检测系统

田毓同 23307110048

2026 年 6 月 28 日

目录

1	设计规划	3
1.1	设计要求	3
1.2	设计思路	3
1.3	设计平台	3
1.3.1	开发板和软件使用	3
1.3.2	设计流程	3
2	设计实现	4
2.1	框图介绍	4
2.1.1	总体状态机	4
2.1.2	顶层系统模块框图	4
2.2	核心模块内部框图	5
2.3	各模块设计和验证（仿真激励和结果波形说明）	5
2.3.1	毫秒计数模块（ms_counter_service）	6
2.3.2	随机等待模块（lfsr_random_service）	6
2.3.3	历史平均模块（score_history_avg）	7
2.3.4	控制器模块（reaction_controller）	7
2.3.5	按键去抖模块（button_debounce_pulse）	8
2.3.6	数码管显示模块（seg7_display_driver）	8
2.3.7	蜂鸣器模块（buzzer_driver）	8
2.4	综合与实现	8
2.5	综合与实现结果	8
2.5.1	综合后资源利用率	8
2.5.2	实现后资源利用率	8
2.5.3	时序结果	10
2.5.4	功耗结果	10
2.6	静态时序结果分析	10

目 录	2
3 FPGA 系统介绍、下载实现和调试，测试游戏过程	11
4 设计总结	11
5 个人体会	13

1 设计规划

1.1 设计要求

本设计实现一个人体反应测试系统，主要要求如下：

1. 使用四位八段数码管显示系统状态与测量结果。
2. 使用 START 与 REACT 两个按键控制测试流程。
3. START 后进入随机等待阶段，显示 ----；随机等待结束后显示反应提示。
4. 记录用户反应时间，分辨率为 1 ms；有效反应范围为 $100\text{ ms} \leq t \leq 5000\text{ ms}$ 。
5. 过早按键或超出有效范围时显示 Fail。
6. 支持显示最近三次有效成绩的平均值。

1.2 设计思路

本设计采用自顶向下的模块化设计方式，将 RTL 分为核心模块与外围交互模块两部分。核心模块负责状态转移、计时、历史记录和随机数生成等内部功能，并向外围显示模块提供数据与状态广播；外围模块接收核心状态并完成数码管显示、蜂鸣器提示，同时把按键控制输入经过去抖后反馈给核心模块。

测试流程由状态机统一调度。用户按下 START 键后，系统进入随机等待阶段，数码管显示等待提示；随机等待时间到达后，系统进入可反应阶段，数码管显示反应提示并启动反应时间计数。若用户在有效时间范围内按下 REACT 键，则系统记录并显示反应时间；若用户过早按键、反应时间小于最小阈值或超过最大阈值，则进入失败状态并显示 Fail。测试结束后再次按下 START 键显示最近三次有效成绩的平均值，再次按 START 开始下一轮测试。

随机等待时间由 LFSR 伪随机序列产生。为避免每次上电后完全固定、可预测的等待序列，随机模块在每次请求随机数时混入自由运行计数器的当前值。该计数器由系统时钟驱动，用户按下 START 的实际时刻会影响采样值，因此即使初始种子固定，每轮测试的等待时间也会受到人工按键时序扰动，从而降低可预测性。

1.3 设计平台

1.3.1 开发板和软件使用

开发板采用 ZYNQ-Z7020 (xc7z020clg484-2)，RTL 综合与实现、仿真工具使用 Vivado 2019.1，并通过 JTAG 下载 bitstream 进行板级验证。项目代码同步整理在 GitHub 仓库 <https://github.com/juruo04/asic>。

1.3.2 设计流程

本项目流程为：先完成模块划分与信号交互协议设计，再分别实现各子模块，随后对各子模块进行独立仿真验证，最后搭建硬件电路、完成下板运行并观察实物结果。

2 设计实现

2.1 框图介绍

2.1.1 总体状态机

该状态机是系统测试流程的总控逻辑，统一描述从空闲、启动、随机等待、有效反应、结果显示到平均值查看的完整闭环。状态机包含 IDLE、PREPARE、WAIT、REACT、FINISH、AVG 和 FAIL 七个状态。

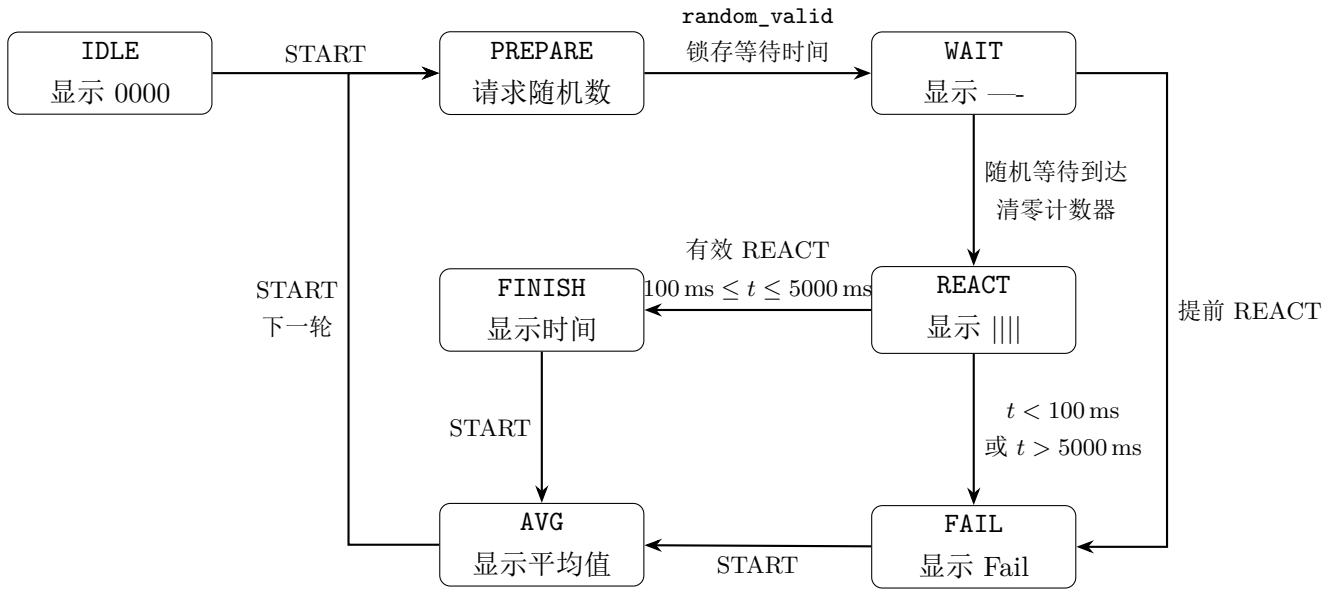


图 1: 反应测试系统总体状态机

2.1.2 顶层系统模块框图

顶层模块 `reaction_system_top` 采用同步时序设计。系统时钟 `clk` 为 50 MHz，复位信号 `rst_n` 为低有效复位信号；二者作为全局同步信号连接到所有时序模块，在框图中统一放置在外侧，不逐条绘制连线。START 与 REACT 两个按键分别进入独立的按键去抖与单脉冲模块，生成 `start_pulse` 和 `react_pulse` 后送入核心模块 `reaction_core`。

在顶层视角下，`reaction_core` 可视为黑盒。它根据按键脉冲完成随机等待、反应时间计数、失败判断和历史平均值计算，并输出状态、失败码、当前反应时间、平均时间以及历史成绩有效标志。数码管显示模块根据这些信号产生段选和位选输出；蜂鸣器模块根据状态和失败信息产生提示音输出。

表 1: 顶层模块功能划分

模块	功能说明
<code>button_debounce_pulse</code>	对 START 和 REACT 两个按键分别同步、去抖，并生成单周期按下脉冲
<code>reaction_core</code>	核心黑盒，完成状态机控制、随机等待、毫秒计数、失败判断和平均值计算
<code>seg7_display_driver</code>	根据核心输出的状态、时间和平均值生成四位八段数码管扫描信号
<code>buzzer_driver</code>	根据核心输出的状态和失败码生成蜂鸣器提示音

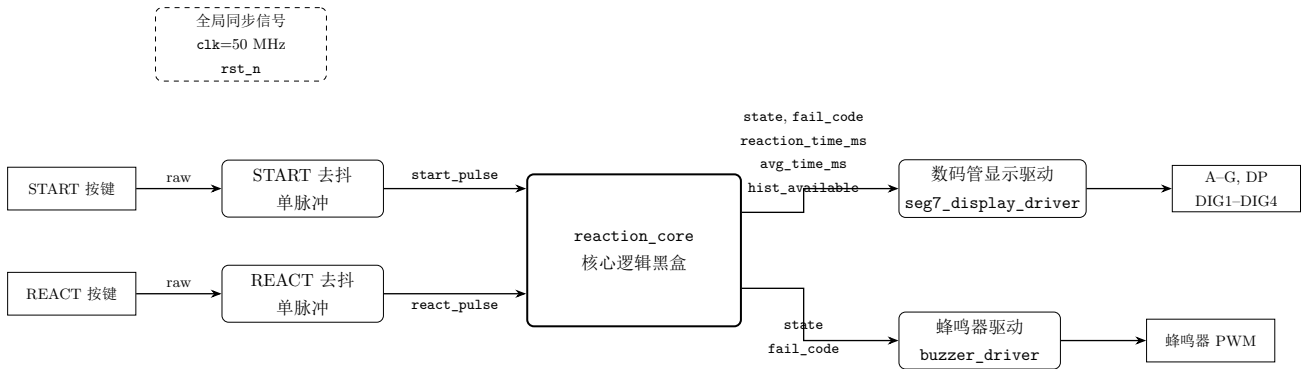


图 2: 顶层系统模块框图

2.2 核心模块内部框图

核心模块是系统功能控制中心。它从顶层接收 `start_pulse` 和 `react_pulse`，内部由控制状态机协调随机等待时间生成、毫秒计数和历史成绩统计。系统时钟 `clk` 与低有效复位 `rst_n` 同步连接到所有子模块，图中以全局同步信号形式标注。

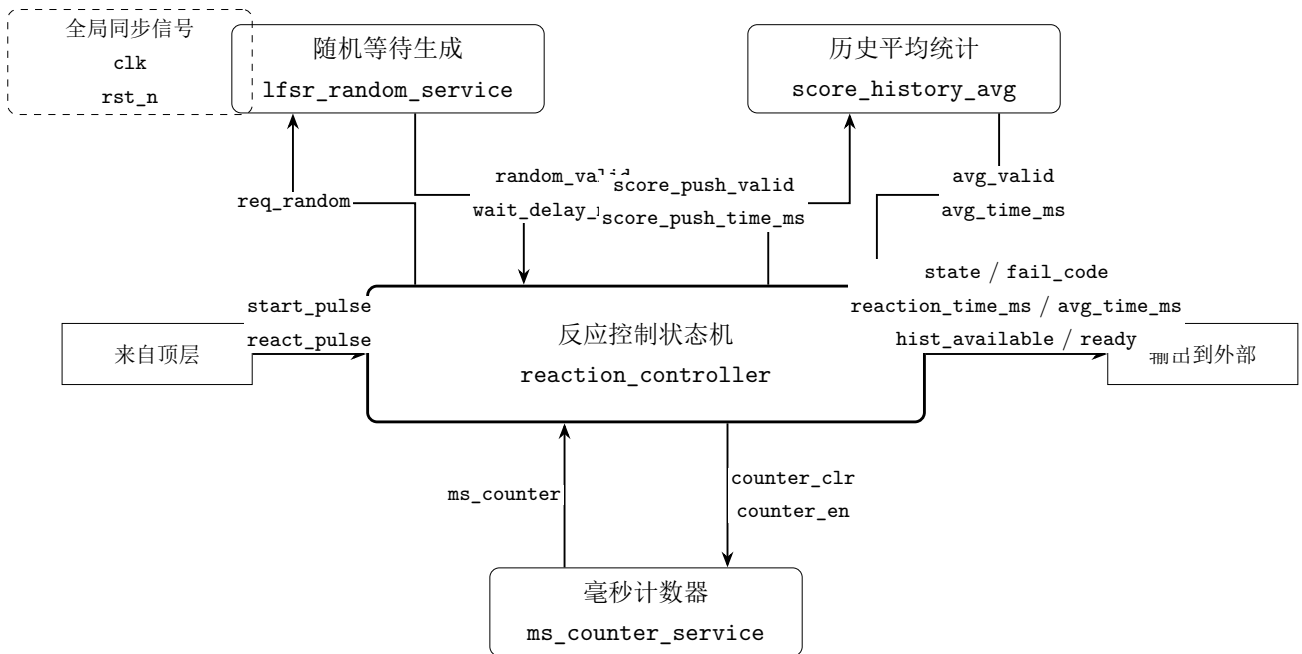


图 3: reaction_core 内部模块通信框图

2.3 各模块设计和验证（仿真激励和结果波形说明）

本系统采用模块化设计方式，将完整反应测试功能拆分为计时、随机等待、状态控制、历史统计和外设输出等若干子模块。各模块之间通过明确的控制信号和数据通路连接，便于分别编写测试平台观察波形。分模块验证时主要关注输入激励、状态或数据变化以及输出响应之间的时序关系，从而确认各子模块功能满足系统设计要求。

表 2: 核心模块内部信号说明

信号	功能说明
req_random	控制器向随机模块请求新的随机等待时间
random_valid / wait_delay_ms_in	随机模块返回有效标志和等待时间
counter_clr / counter_en	控制器清零或使能毫秒计数器
ms_counter	毫秒计数器返回当前计时值
score_push_valid / score_push_time_ms	有效反应完成后写入历史成绩
avg_valid / avg_time_ms	历史模块返回平均值有效标志和平均反应时间

2.3.1 毫秒计数模块 (ms_counter_service)

毫秒计数模块用于为随机等待时间和反应时间测量提供统一的毫秒级计数基准。模块输入包括系统时钟 `clk`、低有效复位 `rst_n`、计数使能信号 `counter_en` 和清零信号 `counter_clr`，输出为当前毫秒计数值 `ms_counter`。

该模块本质上是一个带分频节拍的加法计数器。系统时钟首先被分频形成 1 ms 计数节拍，当 `counter_en` 有效时，计数器在每个毫秒节拍到来时自增；当 `counter_clr` 有效时，毫秒计数值和内部节拍计数均被清零。控制器在进入等待阶段或进入反应阶段时通过清零信号重新开始计时，因此同一个计数模块可以复用于随机等待计时和用户反应计时。

该模块的仿真主要通过波形观察完成。测试平台中先给出复位信号，检查复位后 `ms_counter` 是否回到 0；随后拉高 `counter_en`，观察计数值是否随毫秒节拍递增；最后拉高 `counter_clr`，观察计数是否立即回到 0。仿真波形重点观察以下信号：

1. `rst_n` 释放前后 `ms_counter` 的复位行为；
2. `counter_en` 有效期间计数值的递增过程；
3. `counter_clr` 有效时计数值清零的时序。

2.3.2 随机等待模块 (lfsr_random_service)

随机等待模块用于生成每轮测试开始后的随机等待时间，使用户无法预判反应提示出现的时刻。模块输入为系统时钟、复位和随机数请求信号 `req_random`，输出为随机数有效信号 `random_valid` 以及映射后的等待时间 `random_delay_ms`。

本设计采用线性反馈移位寄存器 (LFSR) 产生伪随机序列。LFSR 每个时钟周期递推一次，其状态更新可抽象表示为

$$x_{k+1} = Ax_k,$$

其中 x_k 为当前 LFSR 状态， A 由反馈抽头决定。模块在收到 `req_random` 请求后，将当前伪随机状态映射到 `[WAIT_MIN_MS, WAIT_MAX_MS]` 区间内，作为本轮随机等待时间输出。为了降低固定初始种子带来的可预测性，模块内部还引入自由运行计数器参与扰动，使用户按下 START 的具体时刻能够影响最终输出。

随机等待模块仿真中每隔 $10\mu s$ 给出一次 `req_random` 请求脉冲。波形显示 `random_valid` 能够在请求后给出有效响应，`random_delay_ms` 随请求更新且处于设定范围内，模块工作正常。

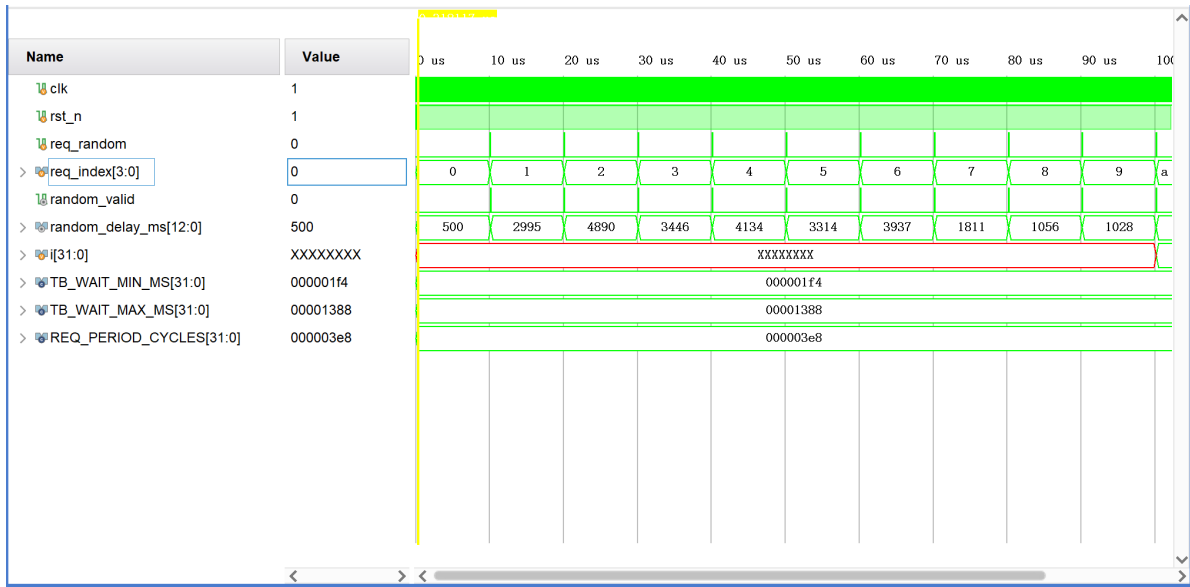


图 4: 随机等待模块仿真波形

2.3.3 历史平均模块 (score_history_avg)

历史平均模块用于保存最近三次有效反应成绩，并计算其平均值供 AVG 状态显示。模块输入包括写入有效信号 `push_valid` 和本次有效反应时间 `push_time_ms`，输出包括平均值有效标志 `avg_valid` 和平均反应时间 `avg_time_ms`。

该模块采用滚动窗口结构保存历史成绩。当 `push_valid` 有效时，模块将本次成绩写入历史队列；若已有三次成绩，则新成绩写入后会淘汰最早一次成绩。只有控制器判定为有效反应并产生 `score_push_valid` 时，历史模块才会更新，因此提前按键、过短反应和超时失败等无效结果不会进入历史平均统计。

该模块仿真时依次输入多组有效成绩，观察平均值是否随样本数量正确变化；随后输入第四组有效成绩，观察最旧数据是否被替换。对于失败场景，控制器不会给出写入有效脉冲，因此历史模块输入端保持 `push_valid=0`，波形上可看到历史平均值不会因失败结果而改变。仿真波形重点观察：

1. `push_valid` 有效时历史数据和平均值的更新；
2. 输入 1、2、3 组成绩时 `avg_time_ms` 的变化；
3. 无效成绩未写入时平均值保持不变。

2.3.4 控制器模块 (reaction_controller)

控制器模块是本系统的状态机核心，负责根据 START、REACT 按键脉冲、随机等待完成标志和毫秒计数值决定系统状态转移，并产生各子模块所需的控制信号。其主要输出包括随机数请求 `req_random`、计数器清零 `counter_clr`、计数器使能 `counter_en`、成绩写入有效 `score_push_valid`、失败码 `fail_code` 和当前反应时间 `reaction_time_ms`。

控制器状态机包含 IDLE、PREPARE、WAIT、REACT、FINISH、AVG 和 FAIL 七个状态。用户按下 START 后，控制器进入 PREPARE 并请求随机等待时间；随机数有效后进入 WAIT，此时计数器开始记录等待时间。若等待阶段提前按下 REACT，则进入失败状态；若等待时间达到随机目标值，则进入 REACT 并重新开始

计反应时间。用户在有效时间范围内按下 REACT 时进入 FINISH 并写入成绩；若反应时间小于最小阈值或超过最大阈值，则进入 FAIL。

控制器仿真采用固定输入激励观察状态转移波形。测试平台中将随机等待时间固定为 1234 ms，在 WAIT 阶段手动使 `ms_counter` 每个时钟周期增加 10 ms，从而压缩仿真时间并清晰展示状态变化。仿真覆盖正常流程、等待阶段提前按键、小于最小反应时间按键以及超过最大反应时间按键四种情况。波形中重点观察 `state`、`start_pulse`、`react_pulse`、`ms_counter`、`counter_clr`、`counter_en`、`fail_code` 和 `score_push_valid` 的对应关系。

2.3.5 按键去抖模块 (`button_debounce_pulse`)

按键去抖模块用于将板上机械按键输入转换为稳定的单周期控制脉冲。

`button_debounce_pulse` 模块用于处理机械按键抖动。原始按键信号首先经过同步寄存器进入系统时钟域，随后通过稳定计数判断按键电平是否已经保持足够长时间。当检测到一次稳定按下时，模块输出单周期 `btn_press_pulse`，供控制器作为 START 或 REACT 事件使用。该模块仿真时通过构造抖动输入和稳定输入，观察只有稳定按下后才产生一个单周期脉冲。

2.3.6 数码管显示模块 (`seg7_display_driver`)

`seg7_display_driver` 模块根据系统状态、反应时间、平均时间和历史有效标志产生四位八段数码管的段选与位选信号。在 IDLE 状态显示 0000；在 PREPARE 和 WAIT 状态显示 ----；在 REACT 状态显示 1111 作为反应提示；在 FINISH 状态显示本次反应时间；在 AVG 状态显示最近三次有效成绩平均值；在 FAIL 状态显示 FAIL。对于数值显示，模块先将二进制时间值拆分为十进制千、百、十、个位，再分别查表生成段码。显示模块仿真时依次切换各个状态，观察段选信号 `seg_a-seg_dp` 与位选信号 `dig1-dig4` 的输出波形是否符合状态分类。

2.3.7 蜂鸣器模块 (`buzzer_driver`)

`buzzer_driver` 模块用于提供声音提示。由于蜂鸣器由三极管和电阻驱动，可按无源蜂鸣器方式输出短时间方波信号。模块检测系统状态是否发生变化，只要 `state` 与上一拍不同，就触发一次固定时长的短促蜂鸣；若状态保持不变，则不会重复触发。仿真时通过依次切换状态观察 `buzzer_pwm`，确认每次状态跳变均产生相同的短时提示波形。

2.4 综合与实现

2.5 综合与实现结果

本设计在 Vivado 2019.1 中完成综合、布局布线和 bitstream 生成，目标器件为 `xc7z020c1g484-2`。实现结果主要来自 Vivado 在实现阶段生成的资源利用率报告、布局布线后时序报告和功耗估计报告。

2.5.1 综合后资源利用率

2.5.2 实现后资源利用率

由表 3 与表 4 可见，综合与实现阶段资源结果基本一致，系统主要消耗少量 LUT、触发器和 I/O 资源，未使用 Block RAM 与 DSP，整体资源占用较低。

表 3: 综合后资源利用率 (synth_1)

Type	Used	Available	Utilisation
Slice LUTs	802	53200	1.51%
Slice Registers	430	106400	0.40%
Block RAM Tile	0	140	0.00%
DSPs	0	220	0.00%
Bonded IOB	17	200	8.50%
BUFGCTRL	1	32	3.13%
MMCME2_ADV	0	4	0.00%
PLLE2_ADV	0	4	0.00%

表 4: 实现后资源利用率 (impl_1)

Type	Used	Available	Utilisation
Slice LUTs	803	53200	1.51%
Slice Registers	430	106400	0.40%
Block RAM Tile	0	140	0.00%
DSPs	0	220	0.00%
Bonded IOB	17	200	8.50%
BUFGCTRL	1	32	3.13%
MMCME2_ADV	0	4	0.00%
PLLE2_ADV	0	4	0.00%

2.5.3 时序结果

表 5: 布局布线后时序结果

Metric	Value
Worst Negative Slack (WNS)	0.065 ns
Total Negative Slack (TNS)	0.000 ns
TNS Failing Endpoints	0
Worst Hold Slack (WHS)	0.102 ns
Total Hold Slack (THS)	0.000 ns
THS Failing Endpoints	0
Worst Pulse Width Slack (WPWS)	9.500 ns
Total Pulse Width Slack (TPWS)	0.000 ns

表 6: 时钟约束摘要

Clock	Period	Frequency
clk_50m	20.000 ns	50.000 MHz

Vivado 时序报告显示所有用户指定时序约束均已满足。根据主时钟周期和 WNS，可估算最大工作频率为

$$F_{\max} = \frac{1000}{20.000 - 0.065} = 50.16 \text{ MHz}.$$

2.5.4 功耗结果

表 7: 布局布线后功耗估计

Metric	Value
Total On-Chip Power	0.123 W
Dynamic Power	0.017 W
Device Static Power	0.106 W
Junction Temperature	26.4 °C
Confidence Level	Medium

综合以上结果，本设计在目标 FPGA 上能够满足 50 MHz 主时钟约束，资源占用和功耗均处于较低水平。

2.6 静态时序结果分析

根据实现后时序报告，主时钟 clk_50m 约束满足，关键指标为 WNS>0、TNS=0，说明设计满足 50 MHz 运行要求。

3 FPGA 系统介绍、下载实现和调试，测试游戏过程

本系统在 ZYNQ-Z7020 开发板上完成验证，START、REACT 和复位按钮使用板上按钮输入，数码管和蜂鸣器驱动电路外接在面包板上实现，其中蜂鸣器通过三极管开关电路进行控制。

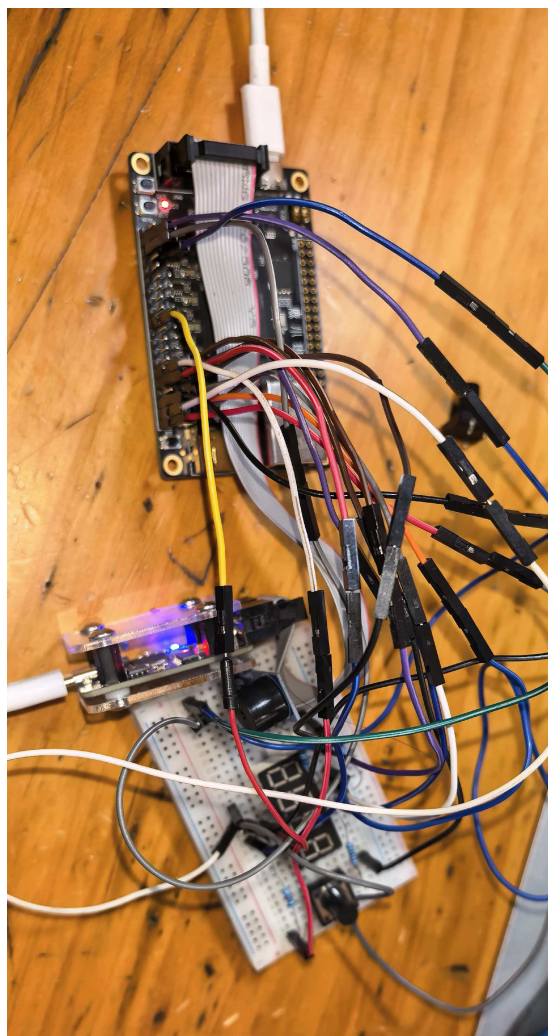


图 5: 开发板与面包板外接电路实物图

完成 bitstream 生成后，通过 JTAG 将设计下载到开发板。上电后按 START 进入随机等待，待显示反应提示后按 REACT 记录反应时间。重复测试可观察 FAIL 判定与平均值显示逻辑。

4 设计总结

本项目完成了从需求到 RTL、仿真、综合实现和板级验证的完整流程。系统结构清晰，资源占用与功耗均较低，能够稳定满足课程要求。

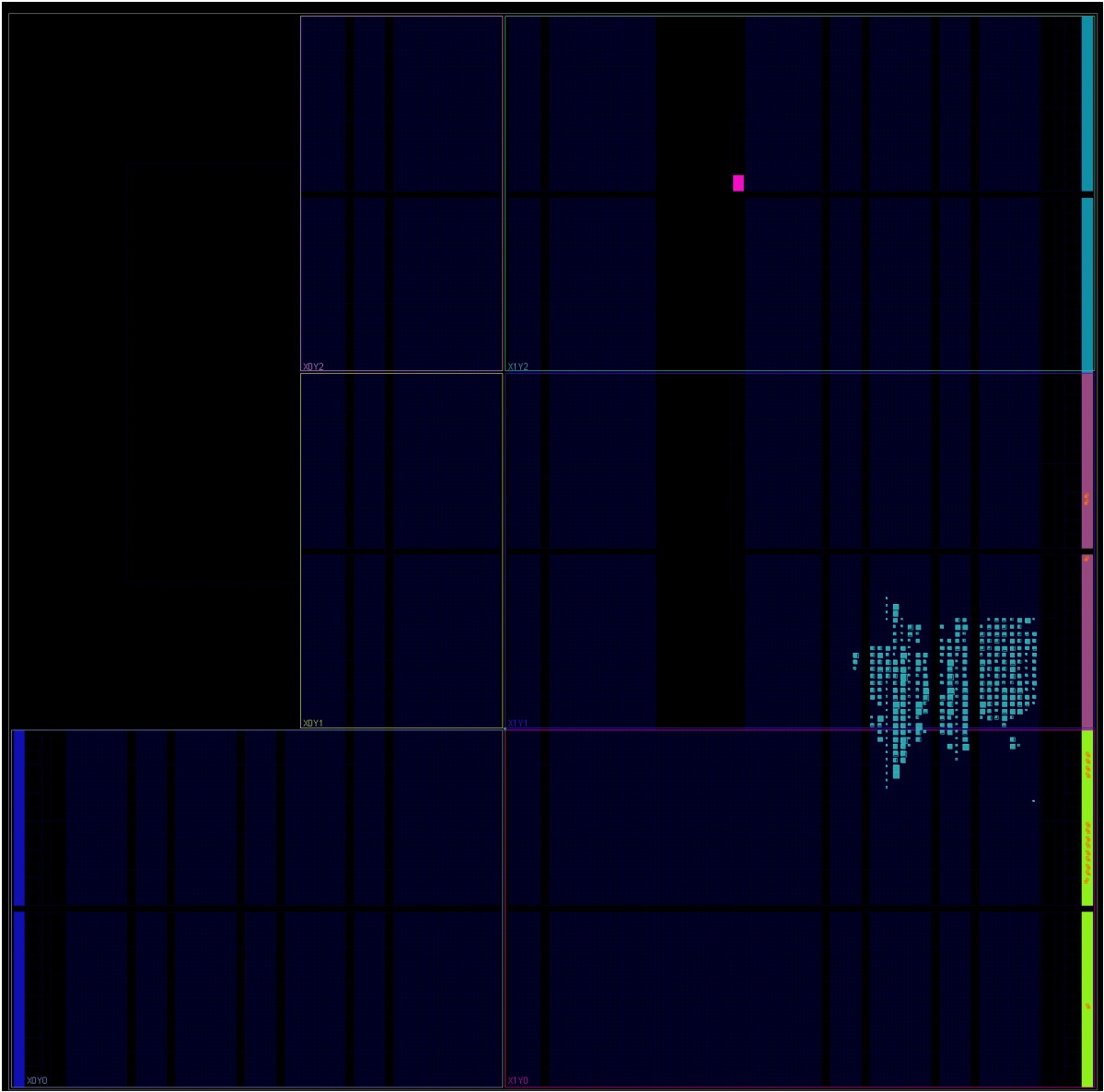


图 6: 顶层连接实线部分的实现视图 (IMP)

5 个人体会

通过本项目，进一步理解了状态机驱动系统设计方法、模块化验证思路以及 FPGA 工程化实现流程。同时在实现阶段体会到约束、时序与板级连线一致性对最终效果的重要性。

附：主要文件说明

文件路径	说明
src/reaction_system_top.v	顶层模块，连接核心、显示、蜂鸣器与按键去抖
src/reaction_core.v	核心模块，集成状态机、随机、计数与历史平均
src/reaction_controller.v	控制状态机
src/lfsr_random_service.v	随机等待时间生成模块
src/ms_counter_service.v	毫秒计数模块
src/score_history_avg.v	历史成绩与平均值计算模块
src/seg7_display_driver.v	数码管显示驱动模块
src/buzzer_driver.v	蜂鸣器提示模块
xdc/reaction_system_gpio1.xdc	引脚与时钟约束文件
sim/*_tb.v	各子模块仿真测试文件